Assignment, WASP Software Engineering Course

Kinga Jenei
Chalmers University of Technology & AstraZeneca

July 8, 2026

1 Introduction

My research explores Large Language Model (LLM) based multi-agent systems for drug discovery, building on an agentic system developed by AstraZeneca [1]. In drug discovery, advancing a molecule toward a clinical candidate involves an iterative design– make–test–analyze (DMTA) cycle, in which scientists design new compounds, synthesize them and evaluate them experimentally. These results are then used to guide subsequent design rounds. This process is supported by computational tools (both rule-based systems and data driven models) assisting with compound generation, evaluation, and synthesis planning. To further support this workflow, AstraZeneca has developed an agentic system called ChatInvent that orchestrates between the various existing AI platforms. Scientists can ask questions using natural language that are answered by the system using the relevant tools.

The system is implemented as a multi-agent architecture in which a supervisor agent coordinates between several specialized sub-agents. A natural language prompt is processed by the supervisor, which selects an appropriate sub-agent and generates a task description for it based on the request. After receiving the output of the sub-agent, the supervisor evaluates the result and iteratively routes subsequent tasks until a final response is produced. Each sub-agent uses an LLM to translate instructions into a sequence of calls to external tools and returns the processed results to the supervisor. Currently, humans have full control over the system. They formulate a hypothesis, and the compounds are generated by giving precise instructions to the agent. Building on this agentic system, my project aims to evolve from the current assistant mode to an AI collaborator that can formulate its own hypothesis and direct its own actions, making the agent much more autonomous.

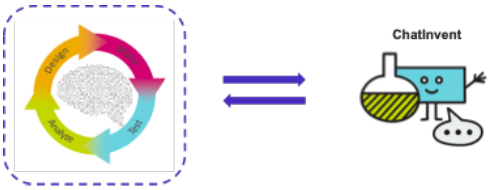


Figure 1

2 Lecture principles

2.1 Group dynamics: the integrated model of group development

One concept from the lectures that I found interesting and relatable to my work is Group dynamics. Robert talked about group maturity stages consisting of 4 parts that can represent an agile team. In stage 1 the team is formed and everything is quite chaotic, then in stage 2 they develop ways of working together but there is no shared understanding of how to do things yet. In the following stage 3 there is structure and trust among the members so the team can work well together. Finally, in stage 4 the team is the most productive and has the least amount of quality issues. The problem, however, is that most organizations are not aware of this model and when there is a reorganization and people are removed and/or new people are added to the team it disrupts the well established group which then goes back to the first stage.

A similar concept based on the above stages can be applied to creating an LLM-based multi agent system as well. In the beginning, the agentic system is very chaotic as it does not have a good structure, and the system prompts are not yet optimized. Then, after some optimization, the agentic system starts to function, but there are no established ways of how to connect the subagents or what is the best way of optimizing prompts. Once these issues are solved and the multi agent system has the correct structure, it performs as expected and can be further optimized using the developed framework to reach the final stage. However, when new agents are added to the team, it can disrupt the well working system and several adjustments might have to be made.

2.2 Cognitive bias

The other concept I choose is Cognitive bias from the Behavioral Software Engineering lecture. In the lecture, we discussed that there are several recurring patterns for individuals that influence their thinking and judgment. One example of this is confirmation bias, meaning that people pay more attention to information that agrees with their views. Another example is availability bias: being influenced by the easily available / recent information.

When working with LLMs it is important to be careful with creating prompts and to be critical about their output. Similarly to confirmation bias, LLMs tend to agree with users rather than challenging them (unless the prompt explicitly asks for it). For example, if the user makes an assumption when asking a question, the LLM will most likely answer the question with the "belief" that the assumption is true. Availability bias can also be found in LLMs as the data on which they were trained on highly influence their output. For example, if something was present in the training data often then the generated answer will more likely include it than something that was present only once in the training data.

3 Guest-lecture principles

3.1 Stakeholder elicitation

The requirements engineering lecture discussed stakeholder elicitation which is the process of listing relevant people (stakeholders) for a project and making the relationships between them explicit. Stakeholders can include people or groups that are users of a system or are affected by it. Regular meetings with the stakeholders are particularly important to introduce them to the system, inform them about any updates or changes and also to discuss their needs and wishes.

This is especially important when developing an agentic system as the underlying LLM can change frequently and the agent can also call tools and make decisions autonomously. Stakeholder requests have to be considered and they should also be well informed about the capabilities and the level of autonomy of the system.

3.2 Hidden gaps become visible

In the guest lecture, Per talked about what happens now that AI is replacing some human tasks. With coordination and interpretation handed over to the agent, it becomes apparent that people do many other things than just their responsibilities. Human roles generally overlap and the responsibility of decision making is fluid. In contrast, agents don't have the same freedom as people which highlights where the gaps are in the organization.

When creating an agentic team, the roles and responsibilities of the subagents are usually well defined and focus only on one specific task. If we take into account the overlap in human roles, these agentic systems should also be developed with a bit more freedom and ample communication between the agents.

4 Data scientists, software engineers, and AI engineers

This section answers questions based on Chapters 1 and 2 of the book *Machine Learning in Production* [2].

What differences do the chapters describe between data scientists and software engineers? Do you agree with this distinction?

In the book, there is a clear distinction between data scientists and software engineers. On one hand, data scientists mainly focus on the data and the machine learning algorithm to build models without taking into account time or cost. They are usually more interested in the research behind the product and experiment with code often written in notebooks. Error handling and code testing is not the main priority, which compared to software engineering practices can be considered as lower quality and harder to maintain over time. On the other hand, software engineers mostly focus on the end product with which users will interact. For them, the time and cost to build a production ready system is very important and the main focus is the user. As they also have to maintain the system over time, software engineers spend more time on testing, error handling and

building a generally more robust system. In general, I agree with this distinction as the two roles have quite different goals and focuses. However, both data scientists and software engineers need to have an understanding and take into account each other to work more efficiently.

The chapters argue that production ML requires a system-wide view, not only a model-centric view. How does this apply to your own research area?

In my research, it is very important to consider the entire system and not just focus on the models behind it. By interacting with users, scientists and engineers, we know what is more important to focus on and the agentic system can be developed in a more efficient way.

Do you think data scientists and software engineers will remain distinct roles, or will both need to learn more of each other's skills?

I think both data scientists and software engineers will need to learn some of each other's skills to work more efficiently and to understand each other better. With technology changing rapidly, it is important to be able to adapt and continuously improve one's skills.

Where does "AI Engineering" fit in this picture: is it mostly software engineering for AI systems, data science made more systematic, MLOps/model engineering, or something partly distinct?

In my view, AI Engineering is data science with an end product in mind. The model is still the focus but by taking into account the final product, workflows are automated and optimized.

How do foundation models, AI coding assistants, RAG systems, prompt pipelines, and agentic workflows change the boundaries between these roles?

With LLM-based assistants and agents becoming more widely used, traditional roles are also changing. The focus of some roles is shifting, there is more overlap between previously distinct roles and new ones are also emerging.

5 Paper analysis

For the analysis, I selected papers from the *5th International Conference on AI Engineering – Software Engineering for AI*.

5.1 RAG-DIVE: A Dynamic Approach for Multi-Turn Dialogue Evaluation in Retrieval-Augmented Generation

The first paper I chose [3] introduces an evaluation framework for multi-turn conversations in Retrieval-Augmented Generation (RAG) systems. Instead of using a static QA dataset, the questions in RAG-DIVE are generated dynamically based on the answers given by the RAG system. This reactive approach ensures that conversations remain accurate and mimic real life scenarios better.

RAG-DIVE includes three components, each powered by an LLM: conversation generator (CG), validator (CV), and evaluator (CE). The evaluation strategy is an iterative process conducted as follows. First, a role and a context document is provided to the CG that generates a question and a ground truth answer along with the question type. Next, these outputs are validated by CV to ensure high quality conversations. After validation, the question is run through the RAG system and the generated answer is fed back to the CG which then generates a follow-up question. This iterative process continues until a predefined number of turns have been made. Once the conversation is over, it is stored and the whole generation process is repeated with new conversations created up to a set number. Lastly, the CE is called which conducts both single- and multi-turn evaluation on all generated conversations.

Evaluating LLM-based agentic systems with question answering capability is a difficult task and has similar challenges to evaluating a RAG system. With multi-turn conversations between the users and the agents being common, it is important to evaluate how the agentic system behaves. In ChatInvent, currently, a set of test questions are evaluated when new features or changes are introduced. However, these questions do not have a ground truth answer. Instead, tool call sequences are evaluated compared to a predefined sequence of tools, and an LLM judge evaluates the final answer. The questions asked by scientists testing the system are also generally more complex than the test question set, and users also ask several follow up questions.

Since multi-turn conversations are not yet systematically tested in our agentic system, implementing a similar evaluation strategy to the approach in the paper described above could be particularly useful. By dynamically generating follow-up questions, we could evaluate the system's behavior and performance in more complex scenarios similar to user interactions. This would be useful in my research to better understand where the system is currently going wrong and what needs to be improved.

5.2 How to Build AI Agents by Augmenting LLMs with Codified Human Expert Domain Knowledge? A Software Engineering Framework

The second selected paper [4] investigates the common problem when companies have only a few experts with specific domain knowledge. This results in too much reliance on a few people and can create problems in decision making. As a proposed solution, the authors developed an autonomous AI agent that captures the domain knowledge of experts and can be used by non-experts to achieve a similar level of outcome. As a selected specialized domain, they focus on the field of visualizing simulation data, where creating plots is challenging as it requires expertise from several domains. Through a systematic process, they develop an agent that generates Python code to create expert-level visualizations based on a natural language input query.

The research process described in the article consists of four steps: 1. extraction of expert knowledge, 2. framework development, 3. system implementation, and 4. validation. First, interviews were conducted with domain experts to capture challenges, knowledge gaps, and any rules that could improve visualizations. These interviews were then thoroughly analyzed and rules provided by the experts were converted to Python functions. Guidelines were generated and incorporated into the system prompt as well as examples for domain specific code generation. In the final system they also incorporated a classifier that routes user queries based on the expert generated rules.

The paper introduces a structured way to capture expert knowledge and convert it to reusable

components to solve complex tasks. This can be useful and important in any AI system where specific domain expertise is required. In ChatInvent, for example, the main expertise required from users is knowledge of chemistry and drug discovery to be able to interpret results and guide the agentic system. In addition to an agent generating new molecules, ChatInvent also has a subagent that analyzes the generated CSV files and can create basic plots using Python. However, the output generated by this Python agent is not always of the expected quality as the agent is not specialized to a specific domain. Moreover, users of ChatInvent are generally computational chemists who are experts in the field of creating new molecules but do not necessarily have computer science or visualization skills. Therefore, they cannot guide the system to generate higher quality analysis or visualizations. By incorporating the ideas from the paper and combining expertise from several domains to "fine-tune" the agent, a more accurate and better quality analysis could be produced by the system.

In my research, creating a system that incorporates expert knowledge is of high importance, and the ideas from this paper could be used to build a more autonomous agentic system.

6 Evidence, originality, and GenAI-use transparency

For the assignment, I relied on the course material provided, Chapters 1 and 2 of the book *Machine Learning in Production* [2], and two selected papers [3, 4] from the 2026 CAIN conference that are available as preprints. I trust both selected papers, as I assume they have undergone peer-review to be presented at CAIN. All sources are listed in the *References* section below.

I have used GenAI tools for some initial brainstorming and occasional language checking, but no LLM-generated text was copied into the assignment.

References

[1] Jiazhen He, Helen Lai, Lakshidaa Saigiridharan, Gian Marco Ghiandoni, Kinga Jenei, Umur Gokalp, Ajsa Nukovic, Ola Engkvist, Jon Paul Janet, and Samuel Genheden. Democratising real-world drug discovery through agentic ai. *Drug Discovery Today*, page 104605, 2026.

[2] Christian Kastner. *Machine learning in production: from models to products*. MIT Press, 2025.

[3] Lorenz Brehme, Benedikt Dornauer, Jan-Henrik Böttcher, Klaus Schmid, Mircea-Cristian Racasan, and Ruth Breu. Rag-dive: A dynamic approach for multi-turn dialogue evaluation in retrieval-augmented generation. *arXiv preprint arXiv:2604.16310*, 2026.

[4] Mikhail Kulyabin, Iris Fuhrmann, Jan Joosten, Nuno Miguel Martins Pacheco, Filippos Petridis, Rebecca Johnson, Jan Bosch, Helena Holmström Olsson, et al. How to build ai agents by augmenting llms with codified human expert domain knowledge? a software engineering framework. *arXiv preprint arXiv:2601.15153*, 2026.